

BLCH9X2 ASSIGNMENT 5- REPORT

Phenyo Thato Molete - 216038155

Github repository: https://github.com/phenyotmolete/BLCH9X2-Block_Chain/tree/assignment5

Part A:

1. Overview:

The previous assignment allowed us to assign a key pair contained within a wallet class to each of our transacting parties, which enabled us to observe how public and private keys work to verify authorship and integrity of transactions. This assignment builds onto the previous. We now have a blockchain class which is made up of an ordered sequence of blocks. Each of the blocks commits to the hash of the block before it, and we see application of the rules which dictate what data is included on to the Chain and verification that the chain has not been silently tampered with once it has been built.

2. Implementation layout:

2.1 The block assembled in assignment 2 is reused:

For this assignment, the demonstration imports Block, create_genesis_block and GENESIS_PREVIOUS_HASH from the block.py created for assignment 2, instead of reimplementing block hashing afresh. The way that a block's hash is calculated is exactly the same as was done in Assignment 2, we see that before calculating the hash, the important data contained in the block is converted into a standardised JSON format, following this, the casing algorithm SHA-256 is run to compute the hash. The information used to calculate the hash includes the index, timestamp, transactions, previous_hash and nonce and of course the stored hash field itself is excluded from the block's own presage.

2.2 The two different tamper detection functions applied to this demonstration:

Function:	Goal:	Criteria:
append_block(block)	This function implements writer protection, ultimately dictating what whether a prospective block meets the criteria to be added to the blockchain.	The function checks: (i) the index: to verify whether the prospect is the correct next block number. (ii) previous_hash to verify whether the prospect correctly points to the current last block. (iii) Self hash to ensure that the block is internally consistent, i.e, if the blocks stored hash matches its actual contents.
Verify_chain(difficulty= None)	This function implements reader protection on the existing blockchain. From the start to end, this function ensures there was no tampering along the chain, after a block was legitimately added to the chain.	The function starts by checking whether the genesis block is correct. Subsequently, the previous_hash link as well as the self-hash of every block within the block-chain is checked. In addition, if difficulty is provided, the function can also verify whether each hash meets the proof-of-work difficulty rule.

3. Execution evidence:

```

-* Checkpoint 1: append_block() writer-protection gates *-
rejected bad previous_hash: bad previous_hash: does not match chain tip
rejected bad index: bad index: expected 3, got 99
rejected bad stored hash: stored hash does not match recomputation
our original blockchain unchanged:still valid, length 3

-* Checkpoint 2: valid chain (length >= 3) *-
length: 3
  Block(index=0, hash=120a20ac98da...) prev=000000000000...
  Block(index=1, hash=ba6a7c351c63...) prev=120a20ac98da...
  Block(index=2, hash=2ae1b91796c4...) prev=ba6a7c351c63...
verify_chain() -> True

-* Checkpoint 3: tamper with Block 1's transaction data *-
Before: Block 1 transactions = [{'sender': 'Phenyo', 'recipient': 'Thato', 'amount': 10}]
After:  Block 1 transactions = [{'sender': 'Phenyo', 'recipient': 'Thato', 'amount': 999}]
verify_chain() -> False
pillar 2 (self-hash) fails: stored hash != recomputation for Block 1

-* Checkpoint 4: recalculate only Block 1's hash *-
verify_chain() -> False
pillar 3 (link) fails: Block 2.previous_hash is stale
  Block 2.previous_hash: ba6a7c351c63c8b1...
  Block 1.hash now:      90e241dddbf626c0...

-* Checkpoint 5: suffix rewrite without proof-of-work *-
verify_chain() [no difficulty] -> True
If we dont assign an adequate difficulty rule, the cheap rewrite is accepted
the fraudulent amount is now taken as 'valid' history in our blockchain.
verify_chain(difficulty=3) -> False
When we enforce an adequate difficulty rule, the same repaired chain we saw before FAILS the verification:
the rewritten hashes were never re-mined, so they don't have the
required 3 leading zero hex digits. Fixing this would
require making the effort to actually re-mine every block from the tamper point to
the tip, this involves real, difficulty-scaled computational work, not just
recomputing a hash.

(For reference: re-mining Block 1 alone at difficulty=3 found nonce=3002, hash=0008a55b416e7c39...)

All checkpoints behaved as expected.
```

3.1 Checkpoint analysis:

Checkpoint	Aim:	Result:	Interpretation:
1	To demonstrate the append_block() function. This checkpoint demonstrates the writer-protection gates.	The function rejects 3 blocks from entering the blockchain. The rejected blocks have an incorrect previous hash, the wrong index and an incorrect/ non matching hash.	The blockchain remains unchanged, the append function rejected each of these blocks from entry into the existing chain.
2	This checkpoint verifies whether the length of the chain, which should contain at least 3 blocks, is valid.	TRUE	The chain passes successfully.

Checkpoint	Aim:	Result:	Interpretation:
3	To demonstrate the <code>verify_chain()</code> function. This checkpoint demonstrates the reader-protection gates by tampering with the data in the first block.	FALSE	When we change the inputs in block 1, we observe that <code>verify_chain()</code> immediately identifies the tampering because block 1's recalculated hash no longer matches the block's stored hash.
4	To demonstrate how recalculating a new hash for block 1 is not sufficient to pass verification again.	FALSE	Remember that each block stores the previous block's hash, so we see here how a person who tries to bypass verification by computing a new hash for block 1 fails because block 2 still contains block 1's old hash, the connection between our blocks is broken.
5	To demonstrate how proof-of-work/difficulty adds an extra layer of protection for our blockchain.	No difficulty = TRUE Difficulty = FALSE	If an unauthorised party rewrites the later blocks, their hashes and their links, they can pass the no-difficulty <code>verification_chain()</code> . But, because the party fails to actually mine new hashes, they are caught when we have a defined chain-level difficulty rule, <code>verification_chain(difficulty = X)</code> .

4. Immutability in a payments context:

4.1 Immutability:

Blockchain immutability does not imply that data cannot be altered. It instead implies that altering data and making the change look legitimate becomes increasingly difficult and expensive.

A hash function cannot physically prevent any party from editing their own local copy of a ledger, if a party has write access, they can still alter a stored transaction and recalculate a new hash for the block they have altered. In this way, we see that immutable does not mean that the data on the blockchain is literally impossible to alter, and we see this from the demonstration at checkpoint 3. The amount within the transaction ledger is changed.

A blockchain does however, offer:

- (I) Detection of tampering: the `verify_chain()` function can, in a length of time proportional to the chain length, recalculate every hash along with every link to check whether anything was changed. Any changes made to a single block are instantly detected as shown in the demonstration at checkpoint 3. And, we see from the demonstration at checkpoint 5 that even if a party attempts to change the data and compute a new internally consistent hash, the chain-level difficulty rule detects this change because they did not actually re-mine the repaired hashes.
- (II) Evading detection is expensive: Once we define a proof-of-work difficulty rule, a party who attempts to repair a block which they have tampered with will now not only have to recalculate the hash, but they will have to search for a new nonce whose hash has the required leading zero hex digits. The cost of searching for the new nonce increases exponentially in line with the difficulty. And to make matters worse, the search for the new nonce has to be repeated for each and every block from the block which was tampered with to the current tip, because each block's hash is contained in the succeeding blocks commitment. So the older the tampered block is on the blockchain, the more remaining work is required of the tampering party.

4.2 Payments context:

So if we think about a payment from Thato to Phenyio of R5000 which was captured and confirmed in block 11, and there are multiple blocks subsequently mined already on top of block 11. A tampering party is able to edit their own private copy of the ledger. The party is also able to re-mine block 11 in isolation, at a low cost and difficulty. However, it would be expensive and difficult for the tampering party to re-mine every block from block 11 to the current tip faster than the honest network extends the real chain further. This is why immutability is economic rather than computational. Even if the tampering party found a way to decrease the cost of doing so, other parties who already hold the honest tip have no reason to discard it for a rewritten alternative unless that alternative represents more cumulative proof-of-work (the longest/heaviest-chain rule). This also explains why banks and remittance desks and exchanges have a policy to wait for a particular number of confirmations before a transaction is finally approved. The security offered by the SHA-256 function is material, but it is defined by computational work and consensus amount all participants within the network, it is not an inherent property of the function itself.

Part B:

1. Overview:

This section uses four illustrative transactions from the Phenyio/Thato ledger, using the same hashing mechanism used in prior assignments (the block info is sorted into a standard, consistent JSON format and encoded using the SHA-256 hashing algorithm). For this assignment, a single digest, `merkle_root()` commits to all transactions in a block, building the tree from the bottom up. At each level, the hashes are pared left-to-right and joined using `SHA256_(left + right)`. For levels where we have an odd number of notes, we follow a similar method to bitcoin, where the last node is paired with a duplicate of itself. This ensures that the paring rule a every level maintains uniformity. When we compute the function `merkle_proof()`, we expect to see the sibling hash and whether it sits on the left or right, and this information is required at each level to enable reconstruction of the root from a single leaf. The reconstruction is performed using the function `verify_merkle_proof()` , enabling us to compare this output with the claimed root.

2. Execution evidence:

```

*Transactions (the four leaves)*
Tx1: {'sender': 'Phenyo', 'recipient': 'Thato', 'amount': 10.0, 'timestamp': 1700000001}
Tx2: {'sender': 'Thato', 'recipient': 'Phenyo', 'amount': 4.0, 'timestamp': 1700000002}
Tx3: {'sender': 'Phenyo', 'recipient': 'Thato', 'amount': 1.0, 'timestamp': 1700000003}
Tx4: {'sender': 'Thato', 'recipient': 'Phenyo', 'amount': 7.5, 'timestamp': 1700000004}

*-Leaf hashes (SHA256 of each canonical transaction)*-
H1 = 5d29c42630357127ecdb665a8877686b35726099a78bd79f96740a17a80b2b87
H2 = 542ac27d01549c0c78a8269ebc8de1868d7c515dbc780b19ee441fac89278a64
H3 = cb7eab88ca0eb76bd91f1d22db4391462d6710a6d828fa573cfc6ad2ded73f7f
H4 = 8a96f30bea7d9b98fd31a915e38cf855390c357220f5308addb6345a87e421b5

*-Level-by-level pairing (hand-calculation trace)*-
H12 = SHA256(H1 + H2) = b51b10eac6388da9d98d14b0233bf28d29b478d787a1c2953b70f9bef2f4e7c2
H34 = SHA256(H3 + H4) = 522aeeee8f36d75128f3785b97082161d399541d298be0720e0be8d2377d15ec
ROOT = SHA256(H12 + H34) = 1a903b25428084d69eb5c2c3c3866d649727815eba2f2b84da0ae7e3ae201b94

merkle_root(LEAF_HASHES) = 1a903b25428084d69eb5c2c3c3866d649727815eba2f2b84da0ae7e3ae201b94
Hand calculation matches merkle_root() output.

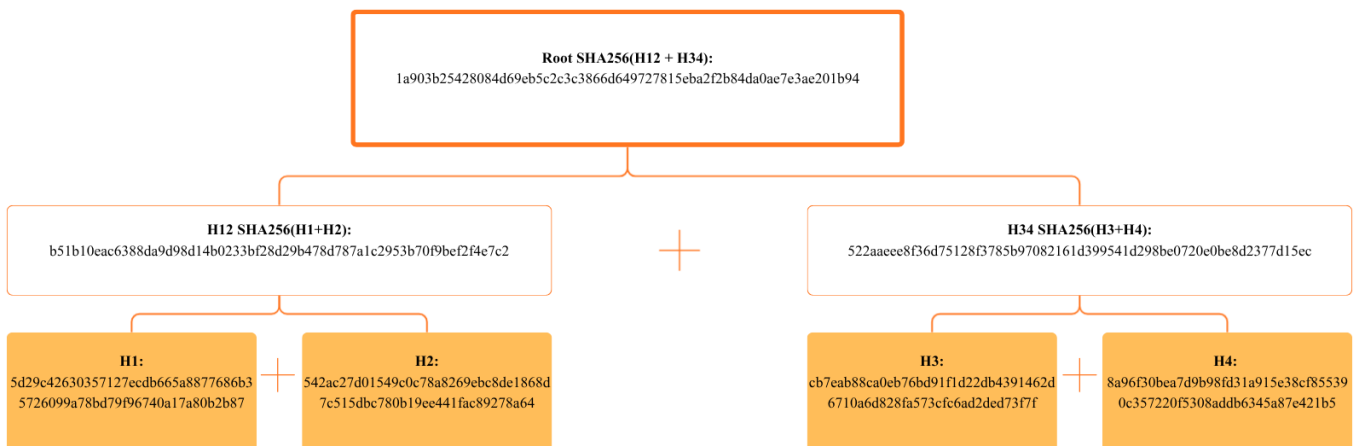
*-SPV proof for H2 (index 1)*-
Proof (sibling, side) pairs: [('5d29c42630357127ecdb665a8877686b35726099a78bd79f96740a17a80b2b87', 'L'), ('522aeeee8f36d75128f3785b97082161d399541d298be0720e0be8d2377d15ec', 'R')]
verify_merkle_proof(H2, proof, root) -> True
verify_merkle_proof(tampered Tx2, proof, root) -> False

All Merkle / SPV checkpoints passed.

```

The tree built from the 4 leaves is structured as:

Merkle root calculation:



The hand calculated trace and merkle_root() function output produce the exact same results, and the provides evidence that the implementation matches the manual pairing rule. When we look at the SPV, the demonstration proves that for H2 we have exactly two sibling hashes. We have H1 which is located to the left and H34 which is located on the right, and is $\log_2(4) = 2$: 2 hashes for 4 leaves. When we re-run the same proof against a tampered version of Tx2 (the amount is changed from 4 to 4000) we see that a different reconstructed root is produced, and this reconstructed root no longer matches, correctly rejecting the tampered transaction.

3. SPV proof analysis:

Simplified payment verification (SPV) is the process which allows a lightweight wallet like a phone wallet, to check whether a a specific transaction actually appears in the blockchain. The wallet does not need to download and validate the entire blockchain, which could be very large, it only downloads the information required for its verification. The wallet does not even need the entire

block containing the transaction, it can use a Merkle proof to verify the transaction more efficiently. The SPV client needs two things:

- (i) A chain of block headers: the block header is a roughly 80 byte summary of the block containing important information including the Merkle root which represents all transactions in that block, the previous block's hash which links it to the blockchain and the proof of work information.
- (ii) A short Merkle proof: This proved that the specific transaction belongs to the Merkle tree represented by that block's Merkle root.

Verification of inclusion is shown in the Merkle tree analysis calculations above, the client hashes the transaction they hold and then makes it way up the tree using the supplied sibling hashes, at each level, the running hash is combined with its given sibling and the output is hashed until we reach a single root. Should the reconstructed root match the the root already committed to in the block header the client is following then the transaction's inclusion in the block is verified cryptographically, without requiring us to trust a third party or to view any other transactions within the block. The client thus separately gains confidence against reversal from how many further headers (blocks) have been mined on top of the block we're looking at. And when we join this analysis with the economic cost argument put forward in part A of this report, this proof-of-work chain of headers is what makes it expensive for the root itself to have been forged or for the block to have been silently altered.

A limitation highlighted in the Mastering Bitcoin text, which also applied to this analysis, is that an SPV proof only establishes inclusion. The SVP proof does not, by itself, establish whether the transaction's inputs were valid or confirm that there was no double spending. To validate deeper history along the blockchain, a full node would still be required. So, what an SPV client trusts here is that the proof-of-work chain the headers are following, represents a material, honest majority of the network. This analysis thus reveals a tradeoff between bandwidth/storage and a weaker trust assumption than one which would come from running a full node.

Part C:

The difference in approaches between the two frameworks doesn't imply conflict, in fact, Nakamoto (2008) and Castro and Liskov (1999) both focus their efforts towards finding a solution for the exact same problem, how does one establish consensus between a group of nodes on history. The key factor driving this analysis is that the approaches start from completely opposite assumptions about who members are (if there are any) and who has is allowed to participate. Nakamoto's framework offers a solution that actually does not require any permission at all. Any party is able to join, as long as they have enough hashing power available to point at the network. Nakamoto's open door policy implies that there's no authentication step to dictate which party counts as a validator and which party does not, so the interesting thing we observe is that nodes compete to propose the next block by solving a Proof-of-Work inequality, which we can denote as $H(m) < D$. What we can observe from this evaluation of Nakamoto's permission-less system is that there really are no measures put in place to stop an attacker from secretly building an alternative chain in parallel. the implication being that in Nakamoto's model, we can never say that finality is absolute, finality is only ever probabilistic. Nakamoto estimates that the probability of an attacker successfully reversing a transaction, denoted as P_z , declines at an exponential rate, as confirmation depth z increases as follows:

$$P_v = 1 - \sum_{k=0}^{\infty} \frac{\lambda^k e^{-\lambda}}{k!} \left(1 - \left(\frac{q}{p} \right)^{\max(z-k, 0)} \right)$$

P_v approximates zero, never truly reaching the zero asymptote. As a result, in practice a user on the system would have to wait for several confirmations before they fully trust that a transaction is settled. This actually shows us the cost associated with Nakamoto's approach of openness: Users benefit since they get censorship resistance, however, the disadvantage is that users forgo speed and certainty.

As said earlier, the PBFT framework's initial assumption is in opposition to Nakamoto's, here Castro and Liskov start by assuming that the validator set is fixed and known in advance. If there are $N \geq 3f + 1$ total nodes, Castro and Liskov's specification only allows up to f of the total nodes to be actively malicious. Their model actually involves no mining, and as a result, consensus is reached via three rounds of messaging (Pre-Prepare, Prepare, Commit) instead.

The implication we can derive from this analysis of Castro and Liskov's model is that finality is therefore deterministic rather than probabilistic. Once a node has collected $2f + 1$ signed commit messages, we have a Quorum Certificate, and the transaction is final, immediately. Here we don't have to wait for proof-of-work, and since every message is signed, if a validator attempts to tamper with the chain, maybe they try to cheat by double-signing, the validator leaves behind unambiguous proof it did so.

If we're looking at an interbank settlement in particular, in my opinion is biased towards the PBFT. I think under conditions of the regulated formal financial sector, Castro and Liskov's specification is the better fit. this is because financial institutions facilitate substantially large transactions and can't professionally tell a client of regulatory body that "the transaction will probably be final after an hour". If we look at settlement finality requirements in the regulated formal financial sector (e.g. CPSS-IOSCO), a definite, legally recognised moment when a transaction is done is expected. And because the banks involved are already licensed, identified, and accountable to regulators, Nakamoto's primary selling point (permission-less access) doesn't actually buy you anything in this setting. So PBFT's instant, deterministic finality wins out over PoW's slower, permission-less model.

References:

Antonopoulos, A.M. (2017). *Mastering Bitcoin: Programming the Open Blockchain*. 3rd ed. Sebastopol, CA: O'Reilly Media.

Castro, M. and Liskov, B. (1999) 'Practical Byzantine fault tolerance', in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*. New Orleans, LA, 22-25 February, pp. 173-186. Available at: <https://pmg.csail.mit.edu/papers/osdi99.pdf> (Accessed: 12 September 2026).

Merkle, R.C. (1980) 'Protocols for public key cryptosystems', in *Proceedings of the 1980 IEEE Symposium on Security and Privacy*. Oakland, CA, pp. 122-134.

Nakamoto, S. (2008) *Bitcoin: A peer-to-peer electronic cash system*. Available at: <https://bitcoin.org/bitcoin.pdf> (Accessed: 12 September 2026).

van Flymen, D. (n.d.) *Learn Blockchains by Building One*. Available at: <https://link.springer.com/book/10.1007/978-1-4842-5171-3> . (Accessed: 12 September 2026).